Matrix operations

Part VII Selected Topics

Scope

Elimination must preserve permutations and earlier multipliers together. Complete LUP leads to inversion, Schur complements, and multiplication reductions. SPD matrices and orthogonal residuals connect Cholesky to least squares while separating exact arithmetic, rank, and numerical sensitivity.

Prerequisites

Linear algebra, Gaussian elimination, and least squares.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Solve systems by LUP and explain permutations.
- Analyze inversion through blocks and LU.
- Handle SPD matrices and least-squares normal equations.
- 28.1 Solving systems of linear equations
- 28.2 Inverting matrices
- 28.3 Symmetric positive-definite matrices and least-squares approximation

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

28.1 Solving systems of linear equations

First use real exact arithmetic. A nonsingular $n \times n$ matrix A has a unique solution to Ax=b for every b. A singular A can give no solution or infinitely many, so an elimination zero does not imply every b is inconsistent. Nonsingularity does not guarantee elimination without row swaps: A=[0 1;1 0] is invertible but its first diagonal pivot is 0.

The LUP contract is **PA=LU**. Matrix P permutes rows, L is unit lower triangular, and U upper triangular with nonzero diagonal. Array p means current row i came from original row p[i], so(Pb)[i]=b[p[i]]. This permutes equations, not columns or unknown x.

From PAx=Pb and PA=LU, solve Ly=Pb then Ux=y. Substitution gives $Ax=P^{-1}Ly=P^{-1}Pb=b$. Nonzero triangular diagonals and invertible P establish uniqueness. Only P=I permits A=LU; assuming that equality while replacing b by Pb without explanation is inconsistent.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Forward and backward substitution dependencies

For unit lower L, process increasing i with $y[i]=b[p[i]]-\Sigma_{j<i}L[i, j]y[j]$. All required earlier y[j] are known, and each step satisfies the first i equations of Ly=Pb.

Process decreasing i with $x[i]=(y[i]-\Sigma_{j>i}U[i, j]x[j])/U[i, i]$. Later unknowns have already been solved, and the last n−i+1 equations of Ux=y now hold. Nonzero diagonals justify division. These invariants complete every row rather than guessing all unknowns simultaneously.

```
Solve(L, U, p, b):
  for i=1..n:
    y[i]=b[p[i]]-sum(j=1..i-1, L[i, j]*y[j])
  for i=n..1:
    x[i]=(y[i]-sum(j=i+1..n, U[i, j]*x[j]))/U[i, i]
  return x
```

Each substitution has n(n−1)/2 summation products; with additions and n divisions, total time is $\Theta(n^2)$, extra vector space O(n). Reuse one factorization for r right sides in $\Theta(n^3+rn^2)$. Refactoring the same A for every b discards reusable work.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Elimination, Schur complements, and existence

For nonzero first pivot a, write the active matrix[a w^T; v B]. Elimination uses

[a w^T; v B]=[1 0; v/a I] · [a w^T;0 B−vw^T/a].

The lower-right S=B−vw^T/a is the Schur complement. The left factor is invertible, so a nonsingular original gives a nonsingular right factor. If nonzero z satisfied Sz=0, vector(−w^Tz/a, z) would lie in that factor's nullspace. Thus S is nonsingular and recursion can continue.

An all-zero pivot column makes the active S singular. Together with previously nonzero pivots, this implies original A singular. Conversely, a nonsingular active S has a nonzero entry in its first column. Swap that entry to the pivot and continue to completion. Hence **every nonsingular A has LUP**, without proving existence of unpermuted LU for every A.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

In-place LUP swaps earlier multipliers too

Store U on and above the diagonal of work, and L's multipliers below it; unit L diagonal is implicit. Swap whole rows, including columns 1..k−1 holding previous multipliers.

```
work=copy(A); p=[1,.., n]
for k=1..n:
  r=argmax(i=k..n, abs(work[i, k]))
  if work[r, k]==0: report singular
  swap(work[k,1..n], work[r,1..n]); swap(p[k], p[r])
  for i=k+1..n:
    work[i, k]=work[i, k]/work[k, k]
    for j=k+1..n:
      work[i, j]=work[i, j]-work[i, k]*work[k, j]
```

Before step k, the block invariant is $PA=[L_{11}\ 0;\ L_{21}\ I]\cdot[U_{11}\ U_{12};0\ S]$. An active-row permutation R changes S to RS and must also change L_{21} to RL_{21}. Otherwise the lower product rows refer to wrong original equations. New elimination multipliers then extend L, preserving PA=LU.

There are $\Sigma_k(n-k)^2=\Theta(n^3)$ trailing-block updates and $\Theta(n^2)$ pivot-search work. These count arithmetic operations, not bit costs for arbitrarily long rationals.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Complete example with two row swaps

Take $A=[0\ 2\ 1;1\ 1\ 0;2\ 3\ 4]$, $b=(0,0,7)^{\mathsf{T}}$. The largest first-column magnitude is 2. Swap rows 1,3, so $p=(3,2,1)$. Multipliers $m_{21}=1/2$, $m_{31}=0$ leave trailing block$[-1/2\ -2;2\ 1]$.

The second column again selects magnitude 2. Swap current rows 2,3, so $p=(3,1,2)$. **Earlier L multiplier 1/2 moves from row 2 to row 3.** New $m_{32}=(-1/2)/2=-1/4$ gives final pivot$-2-(-1/4)\cdot 1=-7/4$.

Thus $L=[1\ 0\ 0;0\ 1\ 0;1/2\ -1/4\ 1]$, $U=[2\ 3\ 4;0\ 2\ 1;0\ 0\ -7/4]$.

Multiplying gives $LU=[2\ 3\ 4;0\ 2\ 1;1\ 1\ 0]$, exactly PA in original-row order 3,1,2. Swapping only the trailing block leaves an erroneous row 2 multiplier 1/2 and breaks LU=PA.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete example with two row swaps (continued)

Two pivots: swap trailing blocks and saved L together

Green: saved L multipliers. Blue: U or the active trailing block. Unit L diagonal is implicit.

Original A

0	2	1
1	1	0
2	3	4

p=(1,2,3)



Swap rows 1/3, then eliminate

2	3	4
1/2	-1/2	-2
0	2	1

p=(3,2,1)



Swap rows 2/3, then eliminate

2	3	4
0	2	1
1/2	-1/4	-7/4

p=(3,1,2)

Swap two moves first-column 1/2 from current row 2 to row 3.

Final m31=1/2, m32=−1/4, U33=−7/4, satisfying PA=LU.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

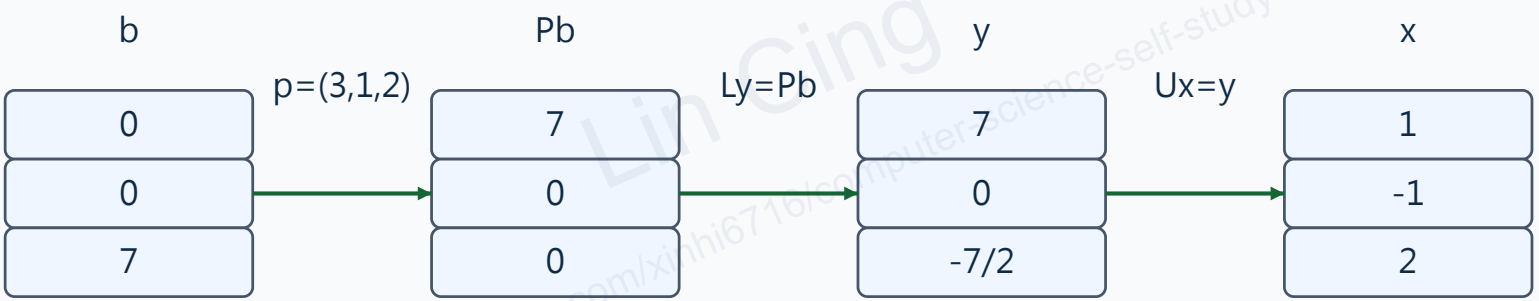
Substitution on the example and numerical boundaries

Here $Pb=(7,0,0)^T$. Forward substitution gives $y_1=7$, $y_2=0$, $y_3=0-(1/2)\cdot 7-(-1/4)\cdot 0=-7/2$.

Backward substitution gives $x_3=(-7/2)/(-7/4)=2$, $x_2=(0-1\cdot 2)/2=-1$, $x_1=(7-3\cdot(-1)-4\cdot 2)/2=1$. Original row checks are $0-2+2=0$,$1-1=0$,$2-3+8=7$.

From b to Pb, y, x: two triangular systems

Same A, b. Row permutation reorders right-side equations, not unknown x.



Forward: $y3=0-(1/2)\cdot 7=-7/2$.

Backward: $x3=2$, $x2=-1$, $x1=1$. Original A gives $(0,0,7)$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Substitution on the example and numerical boundaries (continued)

Partial pivoting selects the largest absolute entry only in the current column. Exact arithmetic guarantees $|m_{ik}| \leq 1$ for this step, limiting column multipliers. It **does not guarantee small floating-point error for every matrix**. Element growth can occur, and ill-conditioning can amplify small input perturbations. Mathematical nonzero pivots differ from implementation tolerance thresholds; near-singularity decisions need scale and error assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

28.2 Inverting matrices

Column j of A^{-1} solves $Ax=e_j$. One LUP plus n substitutions costs $\Theta(n^3)+n\Theta(n^2)=\Theta(n^3)$. Check $AA^{-1}=I$; for square matrices in exact arithmetic, a right inverse is the unique inverse. Solving one $Ax=b$ normally uses substitution directly rather than computing an inverse first.

For $A=[1\ 1;2\ 5]$, $L=[1\ 0;2\ 1]$, $U=[1\ 1;0\ 3]$. For e_1, forward $y=(1,-2)$, then $x_2=-2/3$, $x_1=5/3$. For e_2, $y=(0,1)$, then $x_2=1/3$, $x_1=-1/3$. Thus $A^{-1}=[5/3\ -1/3;-2/3\ 1/3]$, not entrywise reciprocals.

Multiplication checks $5/3-2/3=1$, $-1/3+1/3=0$, $10/3-10/3=0$, $-2/3+5/3=1$. A singular A can still solve selected right sides, but not all unit vectors uniquely, and has no inverse under this definition.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Block-inverse order and assumptions

Write A=[B C; D E], require invertible B, and define $S=E-DB^{-1}C$. Factor A=[I 0; DB^{-1} I] • [B C;0 S]. If A is invertible, so is S. Invert the factors in **reverse order**:

```
A⁻¹ = [ B⁻¹+B⁻¹CS⁻¹DB⁻¹    −B⁻¹CS⁻¹ ]
      [ −S⁻¹DB⁻¹              S⁻¹     ]
```

Matrix factors do not commute. Do not exchange C, D, or inverse positions. In the scalar example B=1, C=1, D=2, E=5, S=3; the top-left term 1+1 • 1 • (1/3) • 2 • 1=5/3 matches columnwise inversion.

This block recursion does not directly cover every invertible A. Matrix[0 1;1 0] is invertible but leading B=0 cannot be inverted. SPD properties below make leading blocks and Schur complements invertible. General matrices require permutations or the following theoretical reduction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Block-inverse order and assumptions (continued)

Schur complement: the exact block after elimination

Left-multiply by $[I\ 0; -DB^{-1}\ I]$. Matrix B must be invertible; factor order matters.

A

B	C
D	E



After elimination

B	C
0	$E-DB^{-1}C$

$S=E-DB^{-1}C$; $A=[I\ 0;\ DB^{-1}\ I]\cdot[B\ C;0\ S]$.

Example $B=1, C=1, D=2, E=5$: $S=3$, $A^{-1}=[5/3\ -1/3;-2/3\ 1/3]$.

For $B=0$, B^{-1} is undefined. Even invertible A can have singular B.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Reductions between multiplication and inversion

For $n \times n$ B, C, construct $3n \times 3n$ T=[I B 0;0 I C;0 0 I]. Its inverse is[I −B BC;0 I −C;0 0 I]; multiplication gives upper-right BC−BC=0. If inversion cost $I(n)=\Omega(n^2)$ and $I(3n)=O(I(n)),\Theta(n^2)$ construction plus one inverse extracts BC in O(I(n)). Dimension and regularity assumptions are essential.

Conversely, first invert SPD blocks. Leading B and Schur S are SPD, as proved next, so invert two half-size blocks and perform a constant number of products:$T(n) \leq 2T(n/2)+O(M(n))$. Assume $M(n)=\Omega(n^2)$, padding below 2n changes only constants, and $M(n/2) \leq cM(n)$ for $c<1/2$. Recurrence levelℓ costs at most(2c)^ℓO(M(n)); the geometric sum gives T(n)=O(M(n)). Powers $M(n)=\Theta(n^{\omega}),\omega \geq 2$ satisfy this.

For general real nonsingular A, Gram $G=A^{T}A$ is SPD because $z^{T}Gz=||Az||^2>0$. Formula $A^{-1}=G^{-1}A^{T}$ follows by multiplying it by A to get I, and retains O(M(n)) cost. Pad nonpowers of two as diag(A, I), whose original inverse block is unchanged. This is an **exact-arithmetic complexity reduction**. Forming $A^{T}A$ solely for this proof is unsuitable as a default floating-point strategy because it squares the condition number. Complex matrices require conjugate transpose A^*, not ordinary transpose.

28.3 Symmetric positive-definite matrices and least-squares approximation

Real SPD means $A=A^T$ and $z^TAz>0$ for **every** nonzero z. Singularity would give a nonzero $Az=0$ and zero quadratic form, so SPD implies nonsingularity. For any leading block B, pad its nonzero vector z to(z,0); positivity of the full quadratic form proves B SPD.

For SPD $A=[B\ C;\ C^T\ E]$, $S=E-C^TB^{-1}C$ is symmetric. For nonzero y, set $z=-B^{-1}Cy$. Then

$(z, y)^TA(z, y)=z^TBz+2z^TCy+y^TEy=y^T(E-C^TB^{-1}C)y=y^TSy>0$.

Thus S is SPD. The first pivot $e_1^TAe_1=A_{11}$ is positive, and elimination recursively handles SPD Schur complements, making every pivot positive. This proves no-pivot elimination avoids zero. Positive original diagonals alone are insufficient:[1 2;2 1] has second pivot−3.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Computing LDL^T and Cholesky

Symmetric elimination gives positive pivots d_j and unit-lower multipliers. Induction gives $A=LDL^T$ with positive diagonal D. Set $C=LD^{1/2}$ to obtain Cholesky $A=CC^T$, with lower C and positive diagonal. A direct column algorithm is

$C[j, j]=\mathrm{sqrt}(A[j, j]-\Sigma_{k<j} C[j, k]^2)$,

$C[i, j]=(A[i, j]-\Sigma_{k<j} C[i, k]C[j, k])/C[j, j]$, for $i>j$.

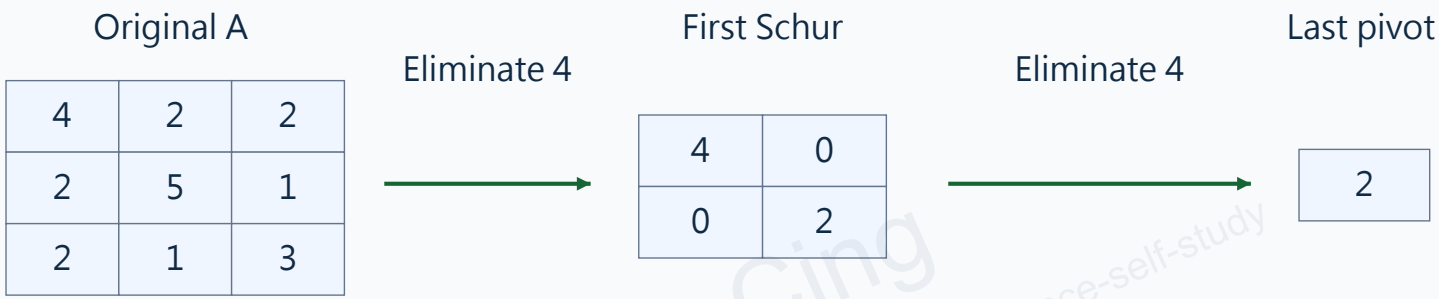
SPD Schur complements keep every square-root argument positive. Triple summations take $\Theta(n^3)$, storing only one triangle. A common flop convention gives leading $n^3/3$, versus about $2n^3/3$ for general LU. This does not promise exactly twice the speed on every machine.

For $A=[4\ 2\ 2;2\ 5\ 1;2\ 1\ 3]$, $C_{11}=2$, $C_{21}=C_{31}=1$, $C_{22}=\mathrm{sqrt}(5-1)=2$, $C_{32}=(1-1\cdot 1)/2=0$, $C_{33}=\mathrm{sqrt}(3-1-0)=\sqrt{2}$. Hence $C=[2\ 0\ 0;1\ 2\ 0;1\ 0\ \sqrt{2}]$. Its product has diagonals 4,5,3 and off-diagonals 2,2,1, matching A.

Computing LDL^T and Cholesky (continued)

SPD Schur complements make Cholesky legal

Every SPD elimination pivot is positive; here 4,4,2.



C rows=[2,0,0]、[1,2,0]、[1,0,√2].

Product CC^T has diagonals 4,5,3 and off-diagonals 2,2,1, matching every original entry.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Least squares: necessity and global sufficiency

For $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, minimize $f(x) = ||Ax - b||_2^2$. Linearity is in coefficients. Basis functions $A[i, j] = f_j(t_i)$ may be nonlinear in t; for example 1, t, t^2 still give a linear coefficient model.

For any direction z, expand $f(x + tz) = f(x) + 2t\, z^T A^T (Ax - b) + t^2 ||Az||^2$. At a minimum, each directional linear term must vanish; otherwise a small t of the opposite sign decreases f. Therefore $A^T Ax = A^T b$. Conversely, if these normal equations hold, setting $t = 1$ gives $f(x + z) = f(x) + ||Az||^2 \geq f(x)$. The stationary point is a global minimum.

Gram $A^T A$ is symmetric positive semidefinite, with $z^T A^T Az = ||Az||^2$. It is SPD **if and only if the columns are linearly independent**. Full column rank gives unique coefficients. Rank deficiency permits multiple coefficients but one prediction Ax:two minimizers differ by z with $Az = 0$. At least one minimizer always exists. Choose independent columns as a basis, solve their full-rank SPD normal equations, and observe that residual orthogonality to the basis gives orthogonality to all columns. For all-zero A, take $x = 0$, $p = 0$, residual b.

Full-column-rank pseudoinverse and projection

For full column rank, define $n \times m$ $A^{+}=(A^{T}A)^{-1}A^{T}$, so $\hat{x}=A^{+}b$. Although $A^{+}A=I_{n}$, generally $AA^{+} \neq I_{m}$. Instead $Q=A(A^{T}A)^{-1}A^{T}$ satisfies $Q^{T}=Q$, $Q^{2}=Q$, and $QAz=Az$ for every column-space vector Az. It projects b onto Col(A), with $A^{T}(I-Q)b=0$.

For the next 3×2 design, A^{+} rows are$(5/6,1/3,-1/6),(-1/2,0,1/2)$, mapping $b=(0,2,1)$ to$(1/2,1/2)$. Q rows are$(5/6,1/3,-1/6),(1/3,1/3,1/3),(-1/6,1/3,5/6)$. This is not I_{3}; it maps b to$(1/2,1,3/2)$, not b itself.

For rank-deficient A, the inverse$(A^{T}A)^{-1}$ is undefined and this formula cannot be applied. The general Moore–Penrose inverse can be constructed by SVD. The formula and coefficient uniqueness here require full column rank.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete fit to three noncollinear data points

Use$(t, y)=(0,0),(1,2),(2,1)$, coefficients $x=(c, m)^T$, and model $\hat{y}=c+mt$.

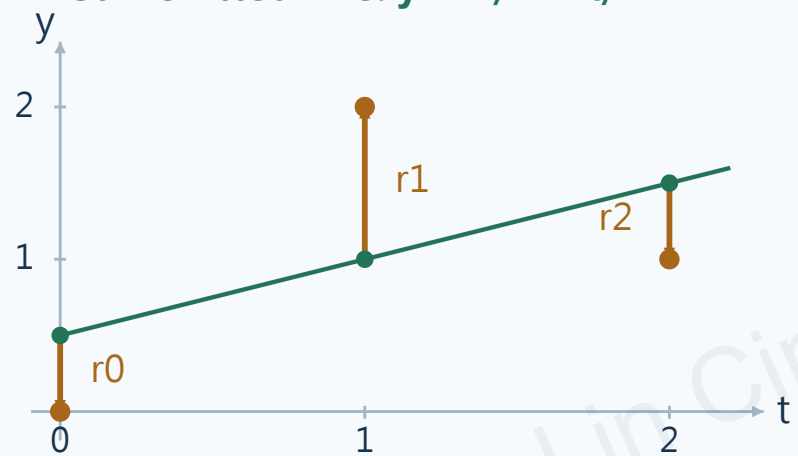
$A=[1\ 0;1\ 1;1\ 2]$, $b=(0,2,1)^T$, $A^TA=[3\ 3;3\ 5]$, $A^Tb=(3,4)^T$. Subtract $3c+3m=3$ from $3c+5m=4$ to get $m=1/2$, $c=1/2$.

t	Observed y	Prediction $\hat{y}$	$r=y-\hat{y}$
0	0	1/2	−1/2
1	2	1	1
2	1	3/2	−1/2

The components of A^Tr are$-1/2+1-1/2=0$ and $0\cdot(-1/2)+1\cdot 1+2\cdot(-1/2)=0$. Squared residual norm is 3/2. Independent columns$(1,1,1),(0,1,2)$ establish unique coefficients. **Collinearity of data points does not determine this design's column rank.** Keeping t and replacing y by$(1,2,3)$ makes the data collinear without changing A; only the residual becomes 0.

Lin Cing
http://github.com/xinhi6716/computer-science-self-study

Complete fit to three noncollinear data points (continued)



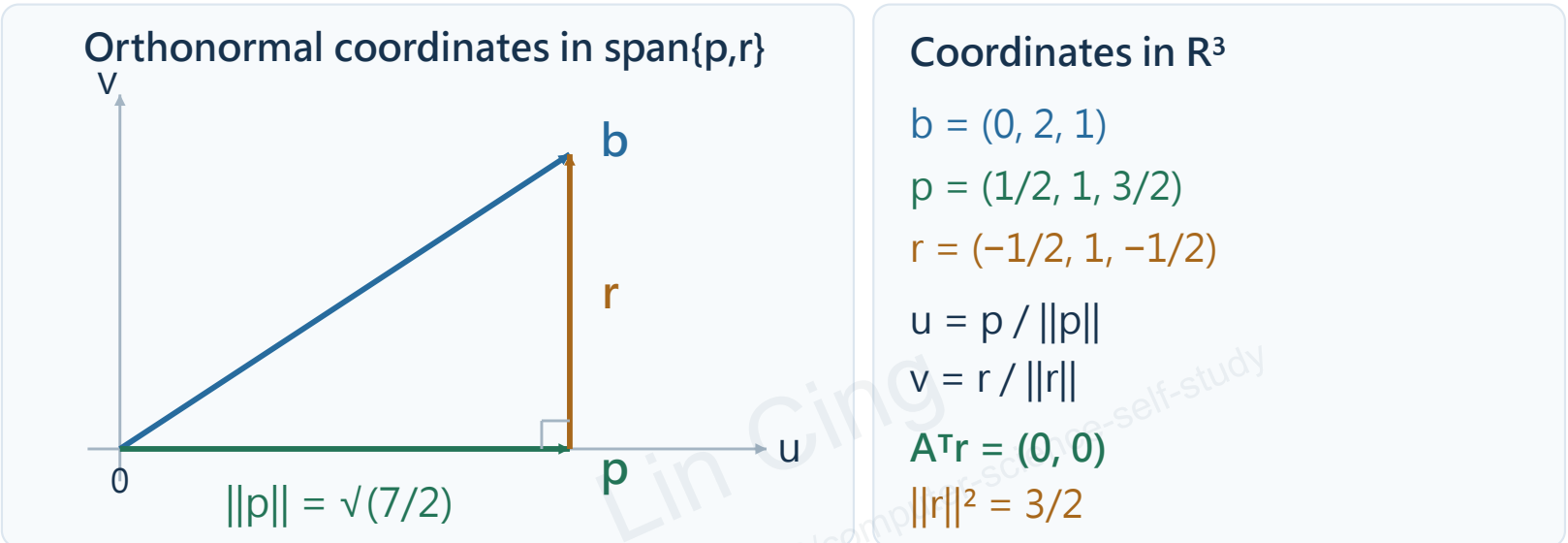
t	y	ŷ	r
0	0	1/2	−1/2
1	2	1	1
2	1	3/2	−1/2

$r^T r = 1/4 + 1 + 1/4 = 3/2$

Amber: vertical error at fixed t. Green: predicted values and fitted line.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Treat the three observations as one vector in R^3



Section E=span{p,r}; its horizontal line is E∩Col(A)=span{p}

For these nonzero orthogonal p,r, use orthonormal axes u=p/||p|| and v=r/||r||. The listed b,p,r retain their original R^3 coordinates. Since $A^Tr=0$, r is perpendicular to all of Col(A). Section E=span{p,r} meets Col(A) only along span{p}; the horizontal line denotes this intersection. Col(A) remains a two-dimensional plane in R^3.

Orthogonal residuals, conditioning, and rank deficiency

Let $p=A\hat{x}$, $r=b-p$. Equation $A^Tr=0$ makes r orthogonal to all Col(A), with $b=p+r$. For coefficients $\hat{x}+z$, the residual is $r-Az$, with orthogonal terms. Squared error is $||r||^2+||Az||^2$, connecting normal equations to minimum-distance geometry.

In the(t, y) scatterplot, r_i is the **vertical y-error at fixed t_i**, not the shortest point-to-line distance. The other picture treats all b as one vector in R^3; there r is the shortest displacement from b to subspace Col(A). These pictures use different spaces.

With repeated columns A=[1 1;1 1;1 1], b=(0,2,1), only $c_1+c_2=1$ is fixed. Every(s,1−s) is optimal, while prediction(1,1,1) and residual(−1,1,0) remain unique.

For full column rank,$\kappa_2(A^TA)=\kappa_2(A)^2$ because singular values are squared. Matrix $\mathrm{diag}(1,10^{-3})$ has condition 10^3, its Gram 10^6. This measures sensitivity, not operation count. QR or SVD often solves floating-point least squares, especially near rank deficiency. Exact normal-equation correctness does not guarantee stability for every floating dataset.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*,3rd ed., MIT Press,2009, Chapter 28.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study